

INSURANCE CARRIER · SOLUTIONS ENGINEERING

Insurance Claims Analyzer

AI-Powered Auto Claims Assessment Platform

Prototype + Production Roadmap

Customer Presentation · 20 min

First-pass damage assessment is the cycle-time bottleneck

4–24h

Current cycle time on simple claims (desk review)

< 5 min

Target time-to-first AI assessment

~15 min

Adjuster touch-time on simple claims today

What we're building: an AI-assisted first-pass assessment that adjusters trust. *Advisory, not authoritative — the licensed adjuster owns every decision.*

✓ Compress cycle time → NPS uplift

✓ Free adjuster capacity for complex claims

✓ Consistent estimates across the book

Single image in. Structured assessment out. ~5–10 seconds.

DEMO FLOW • localhost:3000

1 Click a sample image

Same code path as user upload — no demo cheats

2 Watch the SSE stream

Preprocess → upload → vehicle ID → output gen

3 Three result cards appear

Vehicle / damage / cost — with per-field confidence

4 Show a low-confidence case

Human Review banner + typed triggers

VEHICLE METADATA

Field	Value	Conf.
Make	Honda	95
Model	Civic	92
Year	≈ 2018	60

DAMAGE

MODERATE

Rear bumper deformation, paint scuffing across right quarter panel

REPAIR ESTIMATE

\$2,400 – \$4,800

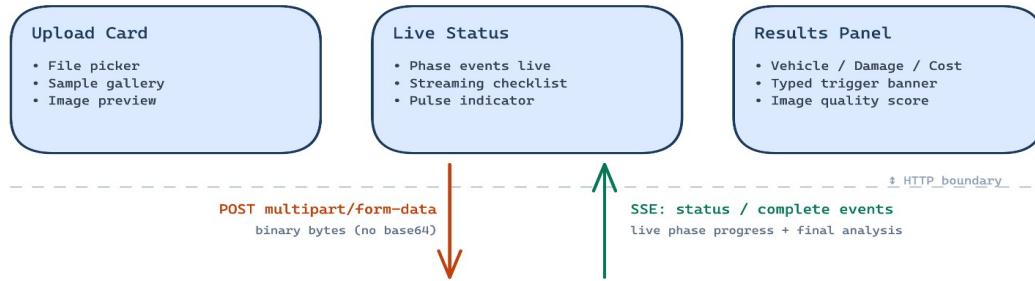
Confidence 72 · Reference table: midrange tier, bumper replacement

One Anthropic Claude call. One vendor. One transaction.

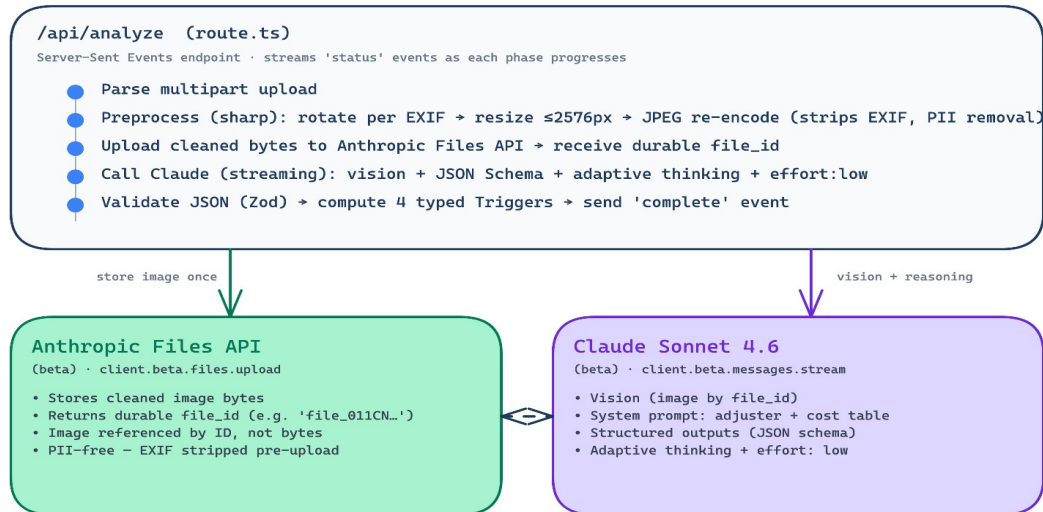
Insurance Claims Analyzer – Architecture

Single Claude API call · Vision + Structured Outputs + Streaming

BROWSER



SERVER (Next.js)



Sample SSE Event Stream

what server emits to the browser as Claude works

```
event: status
data: {"message":"Preprocessing image"}

event: status
data: {"message":"Uploading to Files API"}

event: status
data: {"message":"Identifying vehicle..." }

event: status
data: {"message":"Generating output"}

event: complete
data: {
  "analysis": {
    "car": { "make":"Honda", ... },
    "damage": { "severity":"moderate" },
    "repair_estimate": {
      "low_usd":1200, "high_usd":2800 }
  },
  "triggers": [
    { "code":"high_value",
      "value":11200, "threshold":9000 }
  ],
  "meta": { "model":"claude-sonnet-4-6",
    "file_id":"file_011CN..." }
}
```

Streaming SSE UX

Phase events appear live — wait feels active, not blocked.

Server-side EXIF strip

GPS / device / timestamp removed before any external service sees the image.

Anthropic Files API

Image stored once, referenced by file_id — no S3 plumbing in the prototype.

Structured outputs

JSON Schema enforced at the API level. Frontend never parses model prose.

What's actually in the repo

claims-prototype/

```

├─ app/
│   └─ page.tsx           # Single-page UI; idle/analyzing/done/error
│   └─ layout.tsx        # Root layout (default Next.js)
│   └─ api/analyze/route.ts # The one API route – all backend logic
├─ components/
│   └─ UploadCard.tsx     # File picker + URL input + samples
│   └─ ResultsPanel.tsx   # Three result cards + review banner
│   └─ ConfidenceBadge.tsx # Color-coded confidence pill
│   └─ SeverityBadge.tsx  # Minor/Moderate/Severe pill
├─ lib/
│   └─ anthropic.ts       # Lazy-instantiated Claude client
│   └─ schema.ts          # Zod + JSON Schema (the contract)
│   └─ system-prompt.ts   # Auto-adjuster persona + playbook
│   └─ damage-priors.ts   # Reference cost table (sanity prior)
│   └─ preprocess.ts      # sharp pipeline + URL fetcher
├─ .env.local.example
└─ package.json
  
```

app/

- page.tsx — UI state machine for the demo flow
- api/analyze/route.ts — the entire backend, ~100 lines

One Next.js route IS the backend. SSE streaming endpoint.

components/

- UploadCard, ResultsPanel — main UI surfaces
- ConfidenceBadge, SeverityBadge — calibrated visual cues

Plain React. No framework on top of Next.js.

lib/

- system-prompt.ts — the auto-adjuster persona
- schema.ts — Zod + JSON Schema (the contract)
- preprocess.ts — sharp pipeline + EXIF strip
- damage-priors.ts — Phase-1 cost reference table

Where the AI design decisions live. Phase-1.5 swaps damage-priors for Mitchell/CCC adapter.

Five mechanics that make a single call sufficient

1

Auto-adjuster system prompt

Five-step playbook + inline reference cost table by damage type × vehicle tier (economy / midrange / luxury).

2

Structured outputs (JSON Schema)

Server-validated contract. Every field present, every confidence is an int 0–100. No defensive parsing on the frontend.

3

Adaptive thinking (effort: low)

Claude reasons step-by-step inside the call. Dial up to medium/high for high-stakes claims — same call, deeper reasoning.

4

Server-side typed triggers

Four deterministic rules — low confidence, severe damage, poor image quality, high-value. Bulletproof routing & queue assignment.

5

PII pipeline (sharp + EXIF strip)

Server-side rotate → resize ≤2576px → JPEG re-encode. Drops GPS/timestamps before bytes leave your platform.

Single-call spine + gated chaining

Default: single Claude call (5–10s, one schema, one transaction). *Chained orchestration only as gated specializations where data justifies the extra inference.*

Phase	Chain pattern	Trigger	What chains / Why
1	Cross-validation	high_usd > \$10k	Sonnet 4.6 + Opus 4.7 in parallel → comparison. Independent second opinion.
1	Self-consistency sampling	Confidence < threshold	Re-run Sonnet → variance comparison. Surfaces uncertain assessments.
1.5	Haiku triage tier	All claims (only ungated)	Haiku 4.5 triage → escalate to Sonnet on pass. Cuts blended cost ~30%.
1.5	Parts-pricing fusion	Damage assessment complete	Sonnet → Mitchell/CCC parts API → Sonnet cost grounding.
2	Adjuster co-pilot	Adjuster opens claim	Anthropic Managed Agents API — chained tool/model loop in a session.
3	Fraud pipeline	Similarity above threshold	Hash dedup → vector similarity → AI pairwise → fraud report.

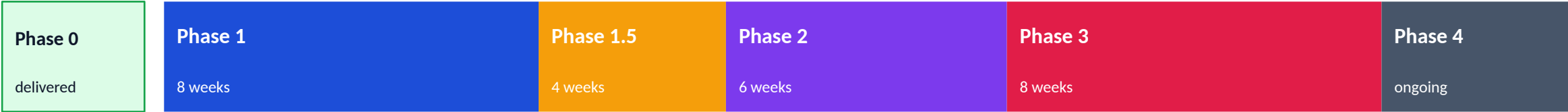
Design rule

Every chain ships with: (a) named failure mode, (b) baseline error rate, (c) trigger condition, (d) A/B-validated metric move vs. single-call. No chain is permanent — quarterly review removes the ones that fail to earn their keep.

Out of scope

Splitting the assessment into vehicle-ID → damage → cost stages (adaptive thinking already does this internally). Generic verifier stages without a named failure mode (2× cost, no real signal — same model agrees with itself).

Phase 0 → 4 · 26 weeks from prototype to fraud pipeline



Prototype	Pilot launch	Cost integration + batch reprocess	Multi-image + co-pilot	Fraud signals	National rollout
-----------	--------------	---------------------------------------	---------------------------	---------------	------------------

PHASE 1 · 8 wks

Pilot launch (production v1)

Hosted multi-tenant, SSO, persistence (Postgres + S3 KMS, 7yr retention), audit log, PII redaction (CV-based plate/face), claims-system integration.

PHASE 1.5 · 4 wks

Cost integration + bulk

Mitchell / CCC ONE / Audatex replaces in-prompt reference table. Anthropic Batches API at 50% cost for non-latency-sensitive workloads.

PHASE 2 · 6 wks

Multi-image + co-pilot

Multi-angle upload, before/after comparison. Adjuster co-pilot on Anthropic Managed Agents API — stateful sessions, PDF/DOCX generation.

PHASE 3 · 8 wks

Fraud-signal pipeline

Image-hash dedup, vector similarity vs. historical claims, AI pairwise on top suspects. Long-running investigations as Managed Agents sessions.

PHASE 4 · ongoing

National rollout + multi-carrier

Hardening, SLA tuning, observability, additional carrier onboarding, mobile-first UX refinements. Per-tenant orchestration config.

Top three risks we're managing — and how

HIGH • HIGH**Adjuster distrust of AI / change-management resistance**

Mitigation: Pilot launches as advisory tool only — no auto-decisions. Confidence indicators + cited sources make every output auditable. Adjuster shadowing in week 1. Calibration gives adjusters control of thresholds.

MED • HIGH**Cost-estimate accuracy below adjuster expectations**

Mitigation: Phase 1.5 replaces reference table with Mitchell/CCC parts pricing. Phase 1 presents cost as a range with confidence + cited sources, framed as 'AI estimate, adjuster reviews'. Confidence-routed escalation.

MED • HIGH**Regulatory disclosure requirements for AI in insurance decisions**

Mitigation: All Phase 1 outputs advisory only — final decisions stay with licensed adjuster. UI clearly labels AI-generated content. Compliance review per state of operation. We follow your legal team's lead on disclosure language.

Full register (10 risks): SOW Section 4 — also covers PII leakage, image quality, API outage, cost overrun, fraud, vendor lock-in.

Numbers we commit to on the pilot cohort

Time to first assessment

< 5 min

from 4–24h

Adjuster touch-time

< 5 min

from ~15 min

Cost-estimate accuracy

≥ 75%

within ±15% of final

Auto-handle rate

30%

from 0%

Adjuster override rate

< 25%

calibration health

Quality & trust

- Confidence calibration: within 10 pp
- Severity-classification: ≥85% agreement with adjuster
- PII-redaction false-negative: < 0.1% (1 in 1000)
- requires_human_review precision: ≥80%
- requires_human_review recall: ≥95%

Operational

- API availability: ≥99.9% (rolling 30d)
- p95 / p99 latency: < 60s / < 90s
- Cost per analyzed claim: < \$0.50
- Audit-log completeness: 100%
- Batches API: ≥45% measured savings vs. real-time

Four things we need from your team

1 Sandbox claims-system access

Guidewire ClaimCenter, Duck Creek Claims, or equivalent. Working integration in week 4.

2 Ground-truth claim dataset

200–500 historical claims with adjuster-final assessments. Calibration dataset + accuracy baseline.

3 Adjuster SME contact

Experienced adjuster for week-1 shadowing and ongoing calibration questions through Phase 1.

4 Branded UI assets

Logo, color palette, type system. Adjuster console feels like part of your platform, not a vendor tool.

Sign-off (Appendix A): Carrier Sponsor • Claims Ops Lead • Solutions Engineering Lead • Implementation Manager

Questions?